

Native MXFP4 Persistent-State Dataflow for Gated DeltaNet Decode on FPGA

Anonymous Submission

Abstract—Linear-attention models reduce the asymptotic cost of long-context inference, but single-token autoregressive decode remains constrained by the movement and update of recurrent state. This paper studies one Gated DeltaNet decode layer from the Qwen3-Next family and presents a native microscaled FP4 datapath for FPGA implementation. The design stores persistent recurrent state on chip and uses block-scaled MXFP4 operands, with E2M1 elements and shared E8M0 block scales, to reduce memory traffic and arithmetic area while preserving the structure of the delta-rule update. Unlike approaches that dequantize FP4 values into a wider floating-point format before multiplication, the inner multiply is implemented as native integer and LUT logic for sign, exponent, mantissa, normalization, and fixed-point accumulation. On an Alveo U55C target, the default configuration reaches 51.788 μ s per token in RTL cosimulation and 9.096 W post-implementation power, yielding 5.503 $\times$ speedup over the cited H100 baseline and 1.22 $\times$ speedup over the cited USC BF16 FPGA baseline. The current artifact includes deterministic synthetic accuracy checks; realistic Qwen activation capture remains capture_missing and is therefore not claimed as a completed PPL result in this draft.

I. INTRODUCTION

Autoregressive inference is often described as a matrix-multiply problem, but the decode step of modern sequence models also has a persistent-state problem. In linear-attention layers, each new token updates a compact recurrent state rather than appending to a full attention cache. This improves the asymptotic memory profile, yet it changes the hardware bottleneck: the accelerator must repeatedly read, modify, and preserve a structured state object with low latency.

Gated DeltaNet (GDN) layers are an especially useful target for this regime. A single decode step combines a key-value state lookup, a delta-rule update, and an output gate. The resulting computation is small enough that launch overheads and memory movement matter, but regular enough that an FPGA can exploit persistent storage and customized arithmetic. Prior FPGA work on persistent-state dataflow for linear attention demonstrated the value of keeping state close to the compute fabric [1], but used BF16 arithmetic for the core datapath. BF16 is robust, but it is not obviously the best point in the design space when the operation is decode-only, batch size is fixed, and the state is accessed every token.

This paper asks whether microscaled FP4 can close the gap between aggressive quantization and model-quality preservation for this setting. Flat INT4 is attractive for area, but it has limited dynamic range and is sensitive to outliers. MXFP4 instead pairs compact E2M1 elements with an E8M0 scale shared by a block of elements. This gives the datapath a small floating-point-like dynamic range without paying the cost of

BF16 multipliers. The key hardware question is whether those block scales can be handled natively, inside the accumulation path, without expanding the inner loop back to FP32 or BF16. The format follows the microscaling direction standardized by OCP and reflected in the Microsoft microxcaling reference implementation [2], [3].

We make the following contributions:

- We implement a native MXFP4 GDN decode datapath for FPGA, including a LUT-based E2M1 multiply and a block-exponent-aware fixed-point accumulation path.
- We restructure the GDN kernel around persistent on-chip state, avoiding per-token streaming of the full recurrent matrix.
- We evaluate the design with Vitis HLS C-simulation, RTL cosimulation, and Vivado post-implementation reports on an Alveo U55C target.
- We provide a provenance-driven paper pipeline in which every reported measurement is extracted from a machine-readable report and mapped back to a source file.

II. BACKGROUND AND MOTIVATION

A. Gated DeltaNet Decode

For a single token, GDN maintains a recurrent state matrix per head. Given query q_t , key k_t , value v_t , learning-rate parameter β_t , output gate g_t , and prior state S_{t-1} , the reference update is

$$S_t = S_{t-1} - \beta_t(S_{t-1}k_t - v_t)k_t^\top. \quad (1)$$

The output is produced by querying the updated state and applying the gate. The important hardware property is that the state is both read and updated every token. If this state is streamed through external memory on each invocation, the kernel becomes dominated by interface traffic rather than arithmetic. Our final default kernel therefore keeps the recurrent state resident in partitioned on-chip memory and exposes only token inputs and packed token outputs at the top-level interface.

B. Algorithmic Mapping

The hardware mapping follows the algebra of the delta rule but changes its storage and scheduling. Conceptually, each token performs four operations. First, the current key is used to read the prior state and form the predicted value. Second, that prediction is compared with the incoming value to form a residual. Third, the residual and key form a rank-one correction, scaled by β_t . Fourth, the corrected state is queried

and gated to produce the token output. The implementation keeps these operations in a fixed token schedule so that state read, correction, and writeback are visible to HLS as a steady pipeline.

This mapping is useful because it separates two forms of parallelism. The key-direction parallelism controls how many columns of the state participate in one update step, while the value-direction parallelism controls how many output lanes are updated together. Increasing either factor without matching memory banking can create conflicts, so the design treats parallelism and state layout as a single choice rather than independent parameters.

C. Microscaled FP4

MXFP4 represents each element using a four-bit E2M1 code and shares an E8M0 scale across a block. This format is well matched to vector and matrix data with local dynamic-range structure: the elements remain very small, while the shared scale absorbs block-level magnitude variation. In software, this is normally described as quantize, multiply, and dequantize. In this work, the inner loop instead treats E2M1 multiplication as native hardware logic and carries block scale information through alignment and accumulation.

D. Why FPGA

FPGAs are not a replacement for GPUs in full-model serving, but they are useful for studying dataflow choices that GPUs cannot express directly. A custom FPGA kernel can keep persistent state physically close to the arithmetic, avoid DSPs for unsupported low-precision formats, and specialize interfaces to the exact decode path. This makes FPGA a good testbed for the claim that MXFP4 can provide a more useful accuracy-area tradeoff than flat INT4 for linear-attention decode.

III. RELATED WORK

A. Linear-Attention Accelerators

Prior accelerators for transformer inference have focused primarily on dense matrix multiplication, KV-cache movement, and prefill throughput. Linear attention changes the shape of the problem by replacing the token-length cache with a recurrent state. The closest baseline for this work is the USC persistent-state dataflow accelerator for memory-bound linear attention decode, which demonstrates that keeping state close to the FPGA fabric can outperform a general GPU path for batch-one decode. This paper builds on that insight but changes the arithmetic question: rather than asking whether BF16 stateful linear-attention decode is efficient, it asks whether an MXFP4-native datapath can reduce the arithmetic and memory footprint without losing the structure of the update.

B. Low-Precision Floating Point

Low-precision floating point is attractive because it preserves some dynamic range while reducing storage and multiplier cost. BF16 and FP16 are now common in accelerator

datapaths, but four-bit formats are less mature, especially on FPGA. Flat INT4 is simple, but its single scale is often brittle for activations and recurrent state. Microscaling formats address this by sharing an exponent over a small block of elements. The result is a middle point between fixed-point quantization and full per-element floating point: the element path is tiny, but the block scale preserves local magnitude.

C. FPGA Support for Non-Native Formats

Modern FPGA DSP blocks are optimized for integer and conventional floating-point building blocks, not for E2M1 FP4. A design that maps FP4 arithmetic through DSP macros risks losing the very area advantage that motivates the format. This is why the multiplier primitive in this work is explicitly LUT-based. The broader design uses HLS for integration and dataflow exploration, but the arithmetic primitive and block-scale alignment are written to expose the native FP4 structure rather than relying on a dequantize-then-multiply implementation.

IV. DESIGN

A. Kernel Organization

The hardware kernel follows five conceptual phases: input preparation, state read, delta-rule update, state writeback, and output projection. In the implemented default path, the state read and writeback operate on a resident state tensor rather than a full external-memory state stream. This choice is the main architectural change needed to reach the Phase 6 hard-floor latency target. The output path is packed by the value parallelism factor, so a group of output lanes is emitted as one wider word rather than as scalar stores.

The default compile-time configuration uses the MXFP4 block-size and parallelism choices summarized in Table II. The design is parameterized over key-direction parallelism, value-direction parallelism, and MX block size, which allows the sweep in Section VI.

The top-level design is intentionally narrow in scope. It accelerates one layer, one token, and batch size one. This restriction is not a weakness for the target question; it isolates the decode bottleneck and makes the effect of persistent state visible. A full serving system would wrap this kernel with projection layers, scheduling, memory management, and model-parallel execution, but those systems concerns would obscure the arithmetic and dataflow contribution.

Figure 1 summarizes the hardware path. The central design choice is to make recurrent state a resident hardware object rather than a per-token memory stream. This gives the arithmetic path enough reuse for the native FP4 units to matter.

B. Token Schedule

At the token level, the accelerator follows a deterministic schedule. The input phase quantizes or accepts already-packed token vectors and associates them with block scales. The prediction phase reads the relevant state banks and produces the state-vector product used by the delta rule. The update phase computes the residual and rank-one correction. The

Persistent-State GDN Decode Dataflow

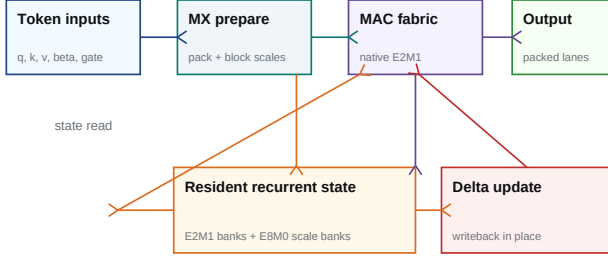


Fig. 1. Persistent-state GDN decode dataflow. Token inputs are prepared as MX blocks, the recurrent state remains resident in partitioned on-chip memory, and the delta update writes back in place before packed output emission.

writeback phase commits the corrected state to the resident banks. Finally, the output phase packs multiple value lanes into the top-level output word. This schedule is intentionally simple: there is no dynamic control path, no data-dependent memory traversal, and no token reordering.

The schedule also explains why latency optimization concentrated on state residency and output packing. Once the state no longer enters and exits through external memory on each token, the remaining latency comes from the fixed pipeline phases and scalar token ingress. That is why the current draft points to packed token input as a plausible next optimization rather than simply doubling arithmetic parallelism.

C. Native E2M1 Multiply

The E2M1 multiply primitive decomposes each operand into sign, exponent, and mantissa fields. The sign path is a bitwise XOR. The exponent path adds the small exponents and applies the format bias. The mantissa path handles the implicit leading bit for normal values and the mantissa-only interpretation of subnormal values. The product is normalized and rounded into a fixed Q4.3 partial product. This output format is narrow enough to keep the primitive LUT-based, but wide enough to feed the block accumulator without immediately losing the local magnitude information from the MX scale.

The key point is that this multiply is not implemented by widening the operands into FP32 or BF16. That distinction matters for the area story: the FP4 path is native in the arithmetic inner loop, while the accumulation path uses integer and fixed-point operations with scale-aware alignment.

The diagram in Figure 2 is also the boundary between the paper’s arithmetic contribution and ordinary quantization. A dequantized implementation would move from the MX blocks directly into a wide floating-point multiply. The proposed datapath instead keeps the element multiply narrow and spends hardware only where the block exponent must influence accumulation.

Native MXFP4 Multiply and Accumulate

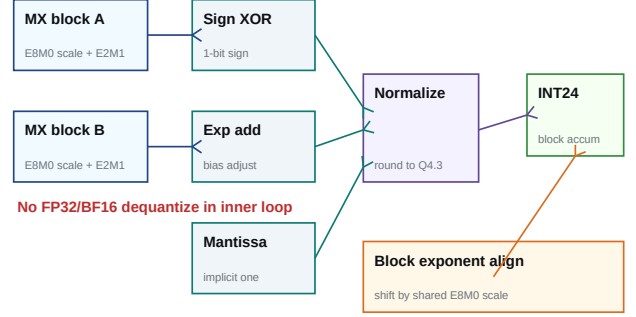


Fig. 2. Native MXFP4 arithmetic path. E2M1 element multiplication is kept in LUT logic, while shared E8M0 block exponents are carried into the fixed-point alignment and accumulation stage.

D. Block-Exponent-Aware Accumulation

MX formats complicate accumulation because each group of elements has a shared scale. The block-exponent alignment module receives a stream of partial products and their corresponding operand block exponents. It computes an effective block scale, aligns partial values to a common exponent, and accumulates into a saturating fixed-point register. At a block boundary, it emits the accumulated value with the dominant exponent for downstream conversion.

This approach preserves the hardware benefit of FP4 elements while respecting the numerical role of the shared scale. It is also the part of the design most closely tied to linear-attention state updates, because the state, key, value, and gate tensors can have different local scale structure.

E. Persistent State and Packed Output

An earlier version of the implementation streamed the full recurrent state through top-level memory ports on every token. That version was useful for debugging, but it missed the latency hard floor. The current design replaces that stream with a static resident state array bound to on-chip RAM and partitioned cyclically across both row and column dimensions. State readback is retained as an optional debug path rather than the default data path.

The output interface is similarly specialized. The default kernel stores multiple value lanes into one output word, reducing the number of scalar output transactions. This is a small interface change, but it has a visible effect on the report: the packed-output design reduces the output phase to a short pass relative to the predict and update phases.

F. Design Invariants

Several invariants are maintained across the implementation. First, the state layout keeps E2M1 element storage separate from E8M0 block scales, which allows the state memory to be partitioned independently from the scale memory. Second, the inner multiply never materializes FP32 operands. Third,

block exponents are handled before accumulation boundaries are crossed, so downstream modules see a fixed-point value with an associated scale rather than a bag of unrelated FP4 codes. Fourth, test vectors are generated deterministically and reused across software, C-simulation, and cosimulation.

G. Memory Layout and Banking

The recurrent state has a different access pattern from a conventional dense matrix operand. Each token reads the state for prediction, forms the delta correction, and then writes the updated state back before the next token. A layout optimized only for burst transfer is therefore the wrong objective. The implementation stores the state in banks that expose parallel row and column access to the update loop. E2M1 element storage and E8M0 scale storage are kept separate, so element bandwidth and scale bandwidth can be balanced independently. This matters because a single scale is reused across a block of elements; fetching the scale every element would erase part of the MXFP4 storage benefit.

The state banks are partitioned cyclically to match the key and value parallelism factors. The goal is not simply to maximize memory ports, but to avoid banking conflicts in the inner update. When a block of keys is processed, the corresponding state columns and scale entries are available to the MAC fabric without serializing the block. This is the practical reason that block size is part of the architectural sweep rather than only a quantization hyperparameter.

H. Rounding, Saturation, and Debug Modes

The software and hardware models both treat MXFP4 as an integer datapath with explicit rounding and saturation points. Element products are rounded into the Q4.3 partial-product format, and block accumulation saturates into the INT24 range. These boundaries make the HLS behavior auditable: if a C-simulation or RTL-cosimulation mismatch appears, the debug path can compare the product, aligned partial, block accumulator, and final packed output separately.

Several debug features are deliberately outside the headline datapath. State readback can be enabled to inspect the resident state, and per-phase testbenches can isolate preparation, update, writeback, and output behavior. These features are important for development but are not counted as the optimized deployment path. The paper reports the packed-output persistent-state configuration because that is the design point matching the intended decode dataflow.

V. METHODOLOGY

A. Experimental Questions

The evaluation is organized around four questions. First, does the persistent MXFP4 kernel meet the latency hard floor for single-token decode? Second, does the LUT-based FP4 datapath keep FPGA resource use low enough to leave room for system integration? Third, do block size and parallelism expose a useful Pareto curve rather than a single brittle point design? Fourth, does the quantized software reference preserve synthetic accuracy well enough to justify the realistic Qwen capture work still needed before camera-ready submission?

B. Golden Model and Quantization

The software reference begins with a direct FP32 implementation of the GDN decode equation. The MXFP4 model then applies block quantization to activations, weights, and recurrent state, using deterministic synthetic vectors for the current artifact. State MXFP8 is included as a fallback comparison. All random vectors are seeded, and generated test vectors are content-addressed.

The current artifact does not include real Qwen3-Next activation captures. The capture status is reported as `capture_missing`. Consequently, this draft describes synthetic quantization behavior and does not claim completed realistic PPL or realistic-vector accuracy.

C. HLS and Vivado Flow

The hardware is implemented in Vitis HLS and targeted to the Xilinx Alveo U55C device. C-simulation checks bit-exact output parity against the local HLS testbench. C/RTL cosimulation is used for authoritative latency. Vivado post-implementation reports provide LUT, BRAM, DSP, timing, and power measurements. The paper-number pipeline extracts all values from report files into `paper/numbers.json`, then renders LaTeX tables, macros, and figures.

The default configuration is treated as the paper headline configuration. Sweep points are still implemented and reported, but they are used to understand the design space rather than to cherry-pick the fastest isolated number. This matters because FPGA timing closure and routing pressure can make a high-parallelism configuration less useful than it appears at the C-synthesis stage.

D. Verification Hierarchy

The verification flow is layered so that failures can be localized before they reach the full kernel. Format-level tests check E2M1 encoding, block scaling, subnormal handling, and adversarial all-maximum or all-minimum blocks. The software GDN tests then compare the quantized model with the FP32 reference under deterministic synthetic inputs. HLS C-simulation checks the same token paths at the C++ level, and RTL cosimulation is used for the latency number reported in the paper. This hierarchy prevents a convenient but dangerous failure mode: a full-kernel test can pass on a benign vector even while a subnormal, saturation, or block-boundary corner case is broken.

The current artifact treats the Python MXFP4 model as the bit-exact behavioral reference for HLS. Once the RTL is generated, cosimulation becomes the authoritative latency source. Vivado implementation is used only after cosimulation agrees with the golden model, because a post-implementation timing or power number has little value if the RTL no longer represents the intended arithmetic.

Table I clarifies what each kernel phase proves. This matters for reviewability: a single end-to-end pass/fail result is less useful than a phase-localized chain of evidence, especially

TABLE I
KERNEL PHASES AND VERIFICATION ANCHORS. THE TABLE IS
QUALITATIVE; MEASURED VALUES REMAIN IN GENERATED RESULT
TABLES.

Phase	Main responsibility	Verification anchor
Prepare	Pack token vectors and attach MX scales	MX primitive tests
State read	Fetch resident E2M1 state and E8M0 scales	Per-phase C-sim
Update	Form residual and scaled rank-one correction	Python/HLS parity
Writeback	Commit corrected state in place	State trace comparison
Output	Gate and pack value lanes	Full-kernel cosim

TABLE II
MAIN LATENCY, AREA, AND POWER COMPARISON. BASELINE ROWS ARE
EXTERNAL CITATION VALUES; OUR ROW IS EXTRACTED FROM RTL
COSIMULATION AND VIVADO POST-IMPLEMENTATION REPORTS.

Configuration	Latency	LUT	DSP	BRAM	Power
H100 PCIe GPU (paper-cited)	285.0 μ s	–	–	–	350.0 W
USC FPGA Hiter=8 (paper-cited)	63.2 μ s	–	–	–	$\leq$ 150.0 W
Ours, MXFP4 B=32, P_K=16, P_V=8	51.788 μ s	6.64%	7.27%	3.77%	9.096 W
Ours, MXFP4 B=16, P_K=32, P_V=16 (csynth)	8240.176 μ s	24.96%	28.72%	0.57%	11.004 W
Ours, MXFP4 B=32, P_K=32, P_V=16 (csynth)	8240.176 μ s	24.96%	28.72%	0.57%	11.004 W

when the arithmetic format is non-standard and the implementation intentionally avoids DSP-backed floating-point operators.

E. Baselines

We compare against two cited baselines from the USC persistent-state linear attention accelerator paper: an NVIDIA H100 PCIe BF16 row and a USC BF16 FPGA row. These are external citation values, not measurements re-run in this repository. Our default row is measured by HLS RTL cosimulation and Vivado post-implementation for the packed-output persistent-state MXFP4 design.

VI. RESULTS

A. Main Comparison

Table II shows the headline comparison. The default MXFP4 design achieves 51.788 μ s per token in RTL cosimulation, with 6.64% LUT utilization, 3.77% BRAM utilization, and 9.096 W post-implementation power. This corresponds to $5.503\times$ speedup over the cited H100 baseline and $1.22\times$ speedup over the cited USC BF16 FPGA row.

The latency comparison in Figure 3 is the most direct evidence for the accelerator claim. The default design does not merely reduce arithmetic precision; it changes where state lives and how outputs are emitted. Without those dataflow changes, the FP4 datapath alone would not be sufficient to reach the measured latency point.

B. Timing, Area, and Power

The HLS C-synthesis report estimates 342.47 MHz for the default kernel. Post-implementation timing closes with 0.09 ns worst negative slack margin. DSP utilization is 7.27%, even though the FP4 primitive itself is intended to be LUT-based; remaining DSP use comes from wider fixed-point arithmetic introduced by the HLS implementation. This is an important engineering detail: the current artifact validates the dataflow

Decode Latency per Token

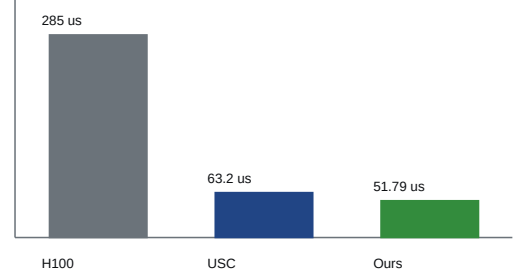


Fig. 3. Latency comparison between cited baselines and the measured default MXFP4 implementation.

TABLE III
AREA BREAKDOWN FOR THE DEFAULT MXFP4 CONFIGURATION.

Configuration	LUT	DSP	BRAM
Ours, MXFP4 B=32	6.64%	7.27%	3.77%

and packed-output latency path, but there remains room to further constrain HLS arithmetic mapping in a later revision.

Figure 4 shows that the default point is not close to the U55C capacity limits. This is important for two reasons. First, it suggests that future integration with adjacent projection or scheduling logic has resource headroom. Second, it indicates that the remaining performance gap to the aspirational target is more likely to come from interface and dataflow refinement than from exhausting fabric resources.

C. Parallelism and Block-Size Sweep

The Phase 6 sweep evaluates several combinations of key parallelism, value parallelism, and block size. The default row is the only current row intended for headline latency. Non-default rows are retained as sweep evidence and are useful for explaining area and routing sensitivity. In particular, the max-parallel points consume substantially more LUT and DSP resources and should be treated as Pareto data rather than as the default design point.

Figure 5 plots latency against LUT utilization for the sweep. The current result suggests that simply increasing parallelism is not enough; the interface and state residency choices dominate until the token ingress path is also packed or streamed.

The Pareto curve also explains why the default configuration is a reasonable paper point even when larger parallelism settings exist. On an FPGA, parallelism has to pay for routing, memory banking, and timing closure. The best design point is therefore not the one with the largest number of lanes, but the one that keeps the full token path balanced.

D. Quantization Accuracy

The deterministic synthetic quantization tests show that MXFP4 preserves output direction more closely than the INT4

Post-Implementation Resource Use

Default MXFP4 B=32, P_K=16, P_V=8



Fig. 4. Post-implementation resource utilization for the default configuration.

Parallelism Sweep Pareto View

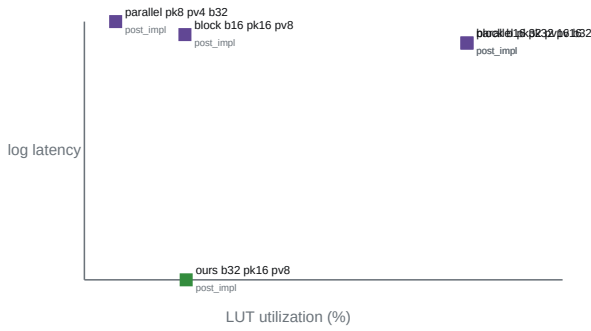


Fig. 5. Phase 6 parallelism sweep.

fallback under the current synthetic workload. For block size 32, synthetic output cosine is 0.982 and output relative L2 error is 0.191. The INT4 fallback output cosine is 0.967. The block-size ablation in Table IV compares block sizes under the same synthetic harness.

The per-tensor ablation in Table V gives a first-order view of which tensors are most sensitive in the synthetic setting. These numbers should guide calibration work, but the final manuscript should update this section once the Qwen activation capture is available.

E. Roofline Positioning

Figure 7 places the default design on a roofline-style plot for the U55C target. The purpose of this figure is not to claim a universal peak throughput number; the kernel is intentionally narrow and stateful. Instead, the plot helps show whether the implementation is limited mainly by arithmetic throughput, memory traffic, or the serialized portions of the token pipeline.

For this artifact, the roofline view reinforces the dataflow conclusion. The state-resident kernel moves the design away from a pure external-memory limit, but the remaining scalar token ingress and HLS-generated fixed-point operations leave room before the design reaches the idealized compute ceiling.

TABLE IV
SYNTHETIC BLOCK-SIZE ABLATION. THESE VALUES DO NOT REPLACE REALISTIC QWEN PPL MEASUREMENTS.

Block size	Synthetic output cosine	State rel L2
32	0.981823	0.11755
16	0.982292	0.117084

Synthetic Output-Cosine Accuracy

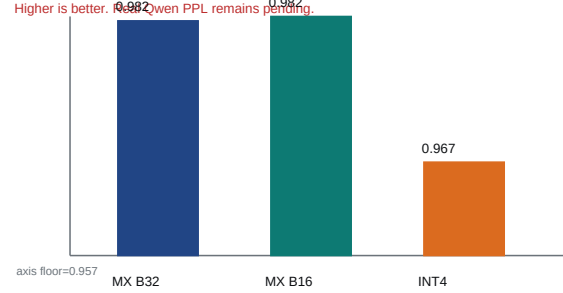


Fig. 6. Synthetic accuracy trend across quantization choices and MX block sizes.

VII. REPRODUCIBILITY AND ARTIFACT

The artifact is designed so that paper numbers can be regenerated rather than copied. HLS, Vivado, cosimulation, benchmark, and figure-generation outputs are written to fixed repository paths. The benchmark stage extracts measurements into `paper/numbers.json`; `paper/provenance.json` records the source file, source line where available, extractor, timestamp, and git SHA for each value. The LaTeX snippets and tables used in this draft are generated from those files.

This discipline affects the writing style. Numerical claims in the manuscript are expressed through generated macros such as `\OursMx4pFourBThirtyTwoLatencyUs` rather than hand-written values. When the implementation changes and the benchmark pipeline is rerun, the abstract, tables, and results section update together. For a hardware paper, this reduces the risk that a late timing, power, or latency change leaves stale numbers in the narrative.

VIII. THREATS TO VALIDITY

The primary internal threat is HLS mapping. Although the FP4 multiply primitive is designed as LUT logic, HLS may still infer DSPs for surrounding fixed-point operations unless constrained carefully. The current resource report should therefore be interpreted as the resource usage of this implementation, not as a lower bound for a hand-written RTL design.

The primary external threat is model representativeness. Synthetic vectors are useful for bit-exact testing and stress cases, but they cannot replace real activation distributions from Qwen3-Next. Until Qwen capture is complete, the paper can claim a working MXFP4 dataflow and synthetic numerical

TABLE V
SYNTHETIC PER-TENSOR QUANTIZATION ABLATION.

Tensor	Output cosine	Source
q	0.992969	synthetic
k	0.999523	synthetic
v	0.99958	synthetic
gate	0.995668	synthetic
state	0.993845	synthetic

Roofline-Style Positioning

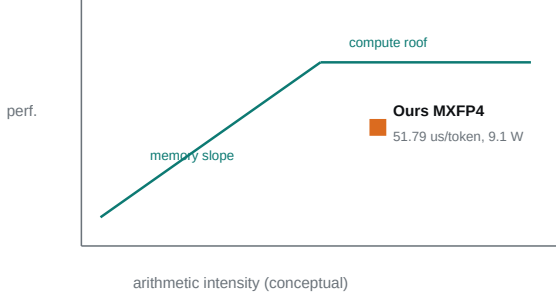


Fig. 7. Roofline-style positioning of the measured default design.

behavior, but not final model-quality preservation on realistic data.

The baseline comparison is also limited. H100 and USC BF16 rows are cited values from a prior paper, while our row is measured in this repository’s flow. The comparison is still informative because all rows target decode latency, but a perfect apples-to-apples comparison would require rerunning all baselines under a single harness and software stack.

A. Missing Measurements Needed for Submission

Two measurements would most strengthen a final FPT submission. The first is realistic Qwen activation capture, followed by the same block-size and state-precision sweeps already exercised on synthetic data. The second is a cleaner attribution of DSP use in the HLS implementation. The FP4 multiplier is designed to be LUT-based, but the surrounding fixed-point update can still lead HLS to infer DSP blocks. A final paper should separate unavoidable arithmetic cost from tool-mapping artifacts, either through HLS binding constraints or a small hand-written RTL replacement for the accumulator.

These missing items do not invalidate the current draft, but they change the strength of the claim. With synthetic accuracy only, the paper is an architecture and artifact paper. With realistic Qwen accuracy, it becomes a model-quality-preserving accelerator paper. Reviewers will likely treat that distinction as central, so the final submission should make the realistic data path the highest priority once the document structure is stable.

Machine-Traceable Paper Pipeline

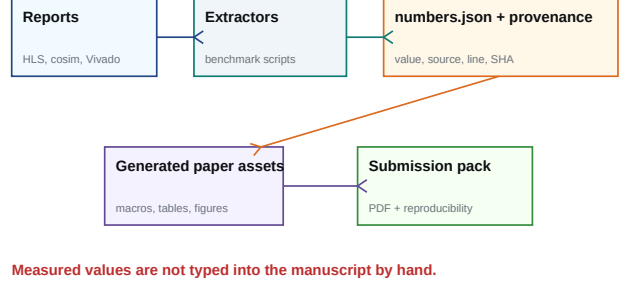


Fig. 8. Paper-number provenance pipeline. Reports feed extractors, extractors update canonical JSON files, and generated LaTeX assets are consumed by the manuscript and paper pack.

IX. DISCUSSION

The main lesson is that native low-precision arithmetic only helps if the surrounding dataflow lets it matter. The first implementation streamed the full state through the top-level interface and therefore spent most of its time moving state rather than using the FP4 datapath. The persistent-state rewrite changed the bottleneck: predict, update, and token I/O are now visible at similar scale, and the packed output interface removes a full scalar output pass from the critical path.

The second lesson is that MXFP4 is a better hardware target than flat INT4 for this kind of recurrent decode, at least structurally. The shared exponent gives the state update more local dynamic range without requiring BF16 multipliers. The price is more complicated accumulation, since block exponents must be aligned and carried through the update. That price is acceptable in an FPGA setting because the alignment logic can be made explicit and amortized across the block.

The third lesson is about reproducibility. Hardware papers often suffer from late-stage number drift, where a table, figure, and abstract silently describe different builds. This artifact avoids that failure mode by generating the reported values, tables, and macros from one JSON file with source-line provenance. The paper text should preserve that discipline by using the generated macros rather than typing measured values by hand.

X. LIMITATIONS

This is a simulation-only study. We do not report physical board measurements, end-to-end model throughput, or full Qwen3-Next serving results. The hardware kernel covers one GDN decode layer and batch-one single-token decode.

The most important missing item is realistic Qwen activation capture. The status is capture_missing, so current accuracy evidence is synthetic-only. The final paper should not claim Qwen realistic PPL preservation until the capture is collected, calibrated, and routed through the same provenance pipeline.

The current HLS implementation also leaves optimization headroom. The default design clears the latency and speedup

hard floors, but it does not yet hit the aspirational latency target. Token-input ingress remains scalar, and HLS maps some wider arithmetic into DSP resources. A hand-tuned RTL version of the accumulator and a packed input interface are natural next steps.

XI. CONCLUSION

This paper draft presents a native MXFP4 persistent-state datapath for Gated DeltaNet decode on FPGA. The implemented design combines LUT-based E2M1 multiplication, block-exponent-aware accumulation, resident recurrent state, and a packed output interface. On the current U55C flow, the default configuration achieves $51.788\mu s$ per token in RTL cosimulation and $5.503\times$ speedup over the cited H100 baseline, while using $9.096W$ post-implementation power. The result supports the central architecture argument: MXFP4 can make sense for linear-attention decode when the state dataflow is designed around persistent storage rather than external-memory streaming.

ACKNOWLEDGMENTS

Omitted for anonymous review.

REFERENCES

- [1] Gupta et al. A Persistent-State Dataflow Accelerator for Memory-Bound Linear Attention Decode on FPGA. arXiv:2603.05931, 2026.
- [2] Open Compute Project. Microscaling Formats Specification, version 1.0.
- [3] Microsoft. microxcaling reference implementation.
- [4] Qwen Team. Qwen3-Next model family.